



Refatoração e desenvolvimento de aplicação para otimização de filas de cirurgias eletivas

Michael Portela Ferreira da Cruz

PROJETO FINAL DE GRADUAÇÃO

CENTRO TÉCNICO CIENTÍFICO - CTC
DEPARTAMENTO DE INFORMÁTICA

Orientadora: Profa. Marley Vellasco - PUC-Rio
Co-Orientadora: Profa. Karla Figueiredo - UERJ

Curso de Graduação em Ciência da Computação
Rio de Janeiro, novembro de 2023

Pontifícia Universidade Católica do Rio
de Janeiro

Michael Portela Ferreira da Cruz

Refatoração e desenvolvimento de
aplicação para otimização de filas de
cirurgias eletivas

Proposta de Projeto Final, apresentado ao
programa Ciência da Computação da
PUC-Rio como requisito parcial para a
obtenção do título de Bacharel em Ciência
da Computação.

Orientadora: Marley Vellasco - PUC-Rio
Co-orientadora: Karla Figueiredo - UERJ

Rio de Janeiro, novembro de 2023

Sumário

1. Introdução.....	1
2. Situação Atual.....	2
2.1. Problema de ordenação de filas de cirurgias eletivas.....	2
2.2. Algoritmo Genético.....	5
2.3. Algoritmo Evolucionário com Inspiração Quântica (AEIQ).....	7
2.4. Algoritmo Evolucionário com Inspiração Quântica para a Área de Saúde (AEIQ-AS).....	8
2.5. Servidor.....	10
2.6. Interface.....	10
3. Objetivos.....	11
4. Atividades realizadas.....	11
4.1. Refatoração do Servidor.....	13
4.2. Refatoração da Interface.....	17
4.3. Análise de sensibilidade.....	27
4.4. Introdução de turnos no AEIQ-AS.....	31
5. Considerações finais e trabalhos futuros.....	34
5.1. Trabalhos Futuros.....	34
5.1.1. Banco de dados.....	34
5.1.2. Tradução do código do modelo.....	35
5.1.3. Uso de dados reais.....	35
6. Referências.....	36

1. Introdução

A demanda por recursos hospitalares é um problema comum em hospitais em todo o mundo, os quais muitas vezes enfrentam a falta de recursos para atender a essa demanda [1]. Essa escassez afeta não apenas os atendimentos de emergência, mas também compromete os procedimentos eletivos, os quais são igualmente cruciais para melhorar a qualidade de vida dos pacientes.

Dessa forma, os procedimentos médicos podem ser classificados de diferentes formas, tendo cada a sua devida importância. Os procedimentos eletivos referem-se a intervenções planejadas, que não são urgentes ou emergenciais, geralmente realizadas para melhorar a qualidade de vida de um paciente, tratando condições crônicas ou oferecer opções terapêuticas não urgentes. Exemplos de procedimentos eletivos incluem cirurgias plásticas, ortopédicas, oftalmológicas, algumas cirurgias bucais, dentre outros procedimentos. Estes são marcados com antecedência, o que permite tanto ao paciente quanto ao médico planejarem-se e prepararem-se adequadamente.

Esses procedimentos são agendados considerando, dentre outras variáveis, a disponibilidade de recursos humanos e materiais. Tanto os recursos humanos, profissionais responsáveis por realizar os procedimentos, quanto os recursos materiais desempenham um papel crucial para viabilizar a realização das intervenções cirúrgicas.

Uma gestão inadequada desses recursos pode resultar na criação de longas filas de espera, que frequentemente são organizadas com base na ordem de chegada dos pacientes, conforme destacado em Nota Técnica da Secretaria de Saúde do Ceará [2]. Entretanto, pacientes que, durante o processo de triagem, foram classificados com estado grave acabam por ter preferência na fila. Ainda assim, diversas outras variáveis são deixadas de lado no gerenciamento dessas filas.

Gerenciar esses recursos é desafiador, visto a enorme quantidade de variáveis que devem ser consideradas. Assim, identifica-se a necessidade de melhorar o gerenciamento das filas e tentar garantir uma melhor utilização dos recursos disponíveis.

Uma possível abordagem para esse problema, é a introdução de métodos de aprendizado de máquina. Esses métodos tem como objetivo organizar a fila de espera levando em consideração múltiplas variáveis, além da ordem cronológica de chegada dos pacientes. Sendo assim, com o auxílio de algoritmos de *machine learning* [3] seria possível além de considerar as

características individuais de cada paciente, usar todas as informações dos recursos materiais e humanos disponíveis, considerando o fluxo de cada processo de atendimento para os pacientes. Isso permitiria a criação de uma ordem de fila que otimiza o tempo de espera, visando atender os pacientes de forma mais eficiente.

O objetivo deste trabalho, é dar continuidade ao desenvolvimento e aprimoramento de uma aplicação que utiliza um modelo de otimização para melhorar as filas de cirurgias eletivas. Por meio de um Algoritmo Genético [3] adaptado, busca-se otimizar o uso de recursos em uma fila de espera para os procedimentos eletivos. Essa ferramenta tem o potencial de reduzir o tempo de espera dos pacientes e melhorar a utilização dos recursos hospitalares, contribuindo para um atendimento mais ágil e eficaz aos pacientes aguardando procedimentos eletivos.

2. Situação Atual

O problema da otimização de filas de pacientes foi inicialmente modelado por meio do Algoritmo Evolucionário com Inspiração Quântica para a Área de Saúde (AEIQ-AS) para otimização das filas, desenvolvido por González e outros [5], baseado em algoritmos genéticos com inspiração quântica. Adicionalmente, no trabalho desenvolvido por Rocha [6], a parte de aplicação foi desenvolvida, incluindo o servidor responsável pela execução e armazenamento dos resultados do modelo, assim como uma interface que busca simplificar o uso do modelo para usuários comuns.

Contudo, como será abordado em capítulos adiante, a forma atual de implementação do servidor, quanto da interface, geram limitações na capacidade de manutenção e desenvolvimento de novas funcionalidades na aplicação atual, sendo necessário reescrever partes da aplicação com o objetivo de melhorar a experiência do usuário e facilitar a extensão de funcionalidades na aplicação.

2.1. Problema de ordenação de filas de cirurgias eletivas.

Antes de iniciar a explicação do funcionamento do modelo é necessário entender como os recursos são considerados para o problema. Conforme mencionado na Introdução, se está lidando com um problema de otimização de ordem para filas de cirurgias eletivas. O modelo busca simular a ordenação de pacientes de acordo com as características de entrada do paciente na fila do

hospital. Na Figura 1, é exemplificado o processo que ocorre após a entrada de um paciente no hospital.

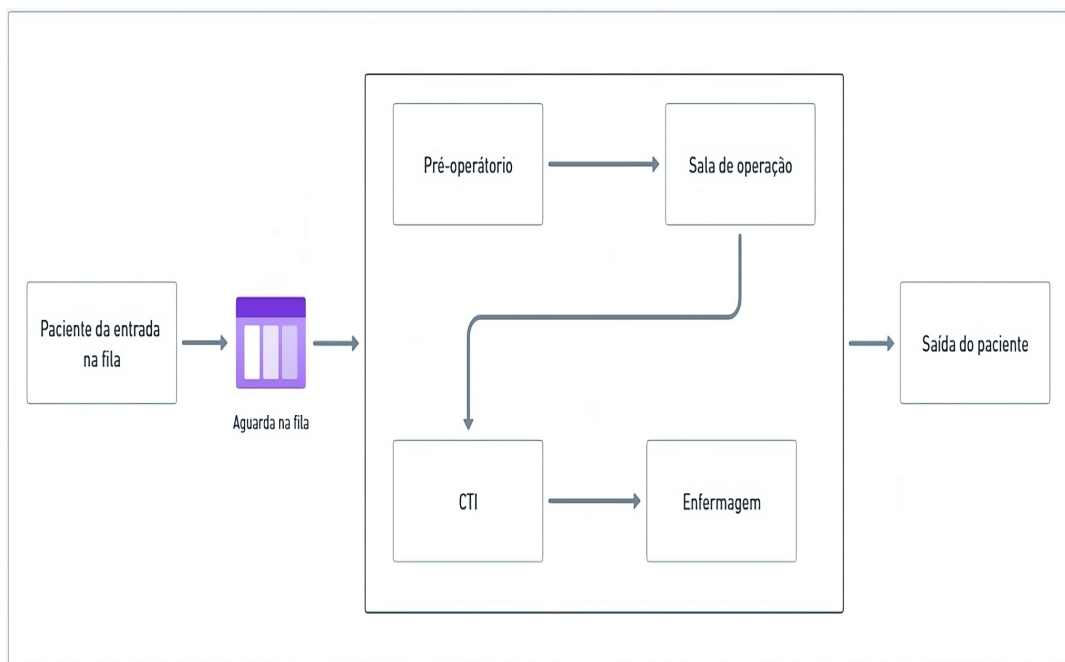


Figura 1 – Fluxo de eventos após a entrada de um paciente

Quando um paciente entra na fila, seu tempo de espera é determinado pela demanda atual. Assim que recursos suficientes estão disponíveis para ele, todas as suas etapas estimadas são agendadas simultaneamente, seguindo a ordem exibida na Figura 1. Não é permitido o agendamento parcial de operações, pois todas elas são agendadas em conjunto. É importante ressaltar que o tempo de permanência do paciente em cada evento é determinado pela gravidade do seu caso específico, e o histórico de pacientes com o mesmo tipo de problema.

Na etapa Pré-operatório o paciente deve passar por procedimentos preparatórios para a sua cirurgia. Seguindo para a Sala de operação, onde o paciente deve ser submetido a sua cirurgia, realizada por uma equipe médica. Como mostrado na Figura 2, a equipe médica é formada por um Médico Especialista, um Assistente e um Anestesista. A quantidade de cada membro da equipe e a duração total da operação variam de acordo com a complexidade do procedimento necessário para o paciente, mas a equipe deve contar com no mínimo um médico especialista, um assistente e um anestesista. Após a intervenção cirúrgica, o paciente é levado para o Centro de Terapia Intensiva (CTI) e, posteriormente, transferido para a Enfermaria, onde deve se recuperar até sua saída do hospital.



Figura 2 – Equipe mínima presente em uma Sala de operação

Na Figura 3 consegue-se visualizar a divisão entre os tipos de recursos considerados no modelo. Além do descrito na imagem, tanto o recursos materiais quanto recursos humanos contêm suas próprias especialidades. Por exemplo, se um paciente entra na fila necessitando de uma cirurgia específica que demanda uma especialidade em particular, apenas recursos que tenham essa especialização específica poderão ser alocados para o atendimento. Caso não haja disponibilidade de recursos com essa especialização, o paciente precisa aguardar até que tais recursos estejam disponíveis para seu tratamento.

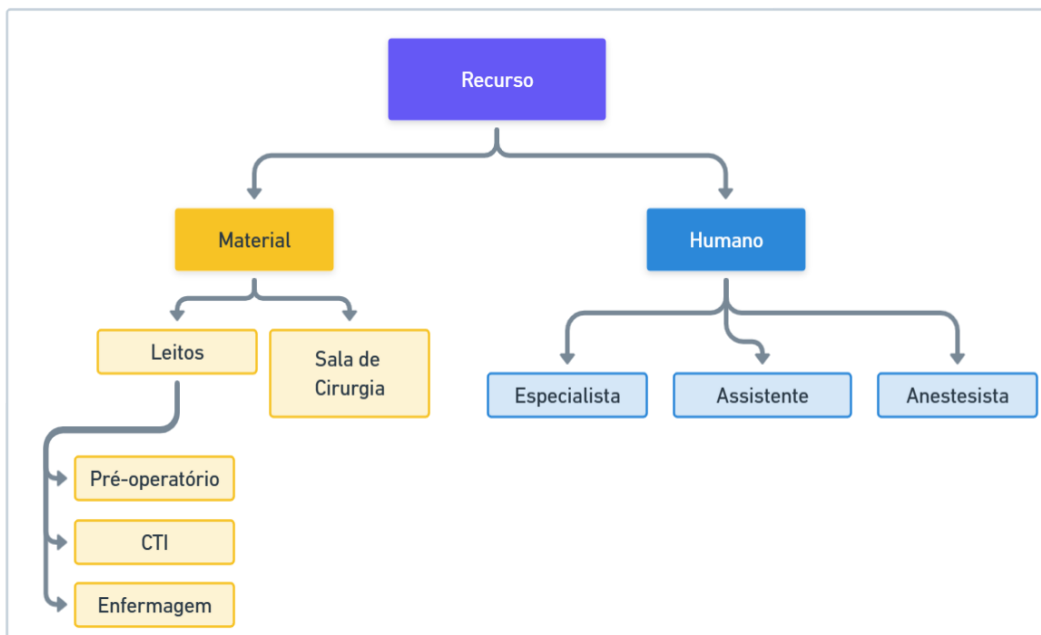


Figura 3 – Diagrama com os recursos considerados

Além disso, outro ponto que o modelo deve levar em conta é a prioridade de cada paciente. Em González e outros [4], os critérios levados em conta são: a

espera pela cirurgia pode acarretar na piora do estado do paciente e a existência de dor no paciente. A Figura 4 ilustra o processo para a decisão da prioridade do paciente.

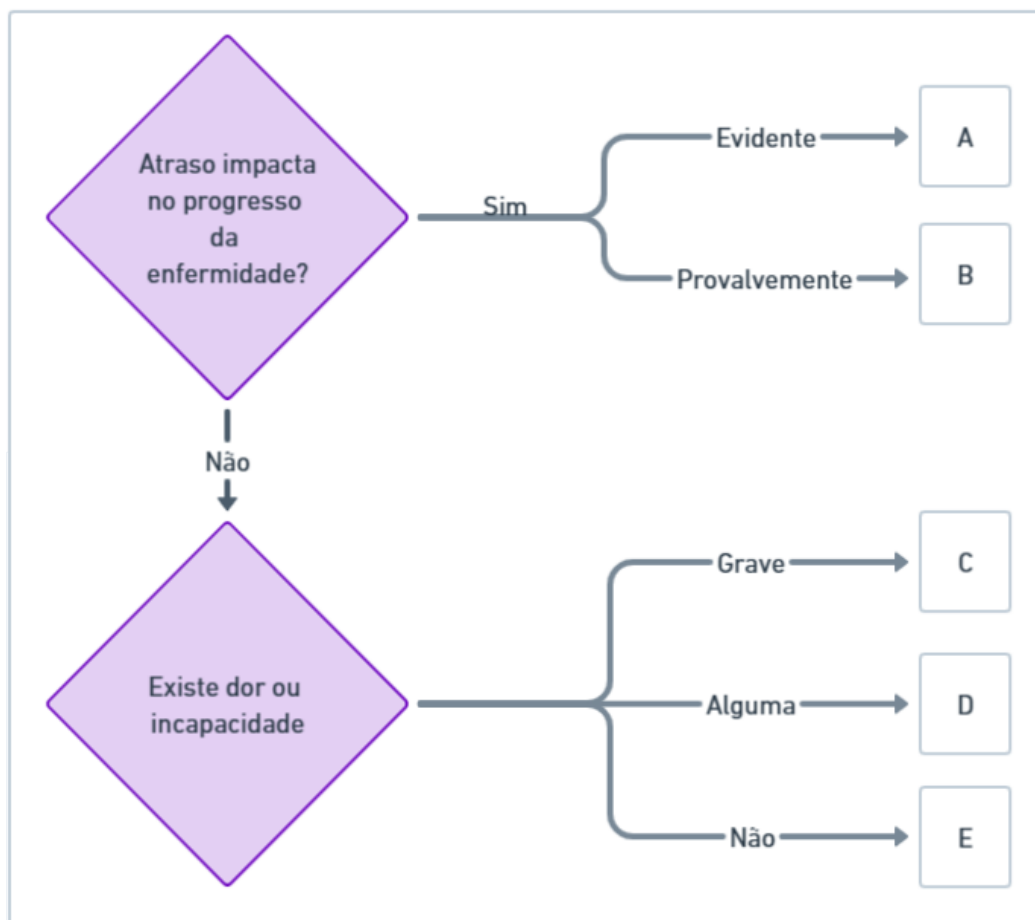


Figura 4 – Diagrama decisão de prioridade do paciente

2.2. Algoritmo Genético

Os Algoritmos Genéticos [3], são métodos de otimização inspirados no processo de evolução natural, utilizando técnicas baseadas no processo de seleção natural, mutação e hereditariedade. São amplamente utilizados para encontrar soluções aproximadas para problemas complexos de otimização e busca.

Na Figura 5 pode-se acompanhar as etapas de um algoritmo genético clássico. De forma resumida, o algoritmo inicia com a geração da população inicial, onde cada indivíduo é representado por um conjunto de parâmetros (genes). Ao longo das iterações, essa população é avaliada conforme um critério de aptidão, cujo objetivo é determinar quais indivíduos são mais aptos para sobreviver e consequentemente se reproduzir.

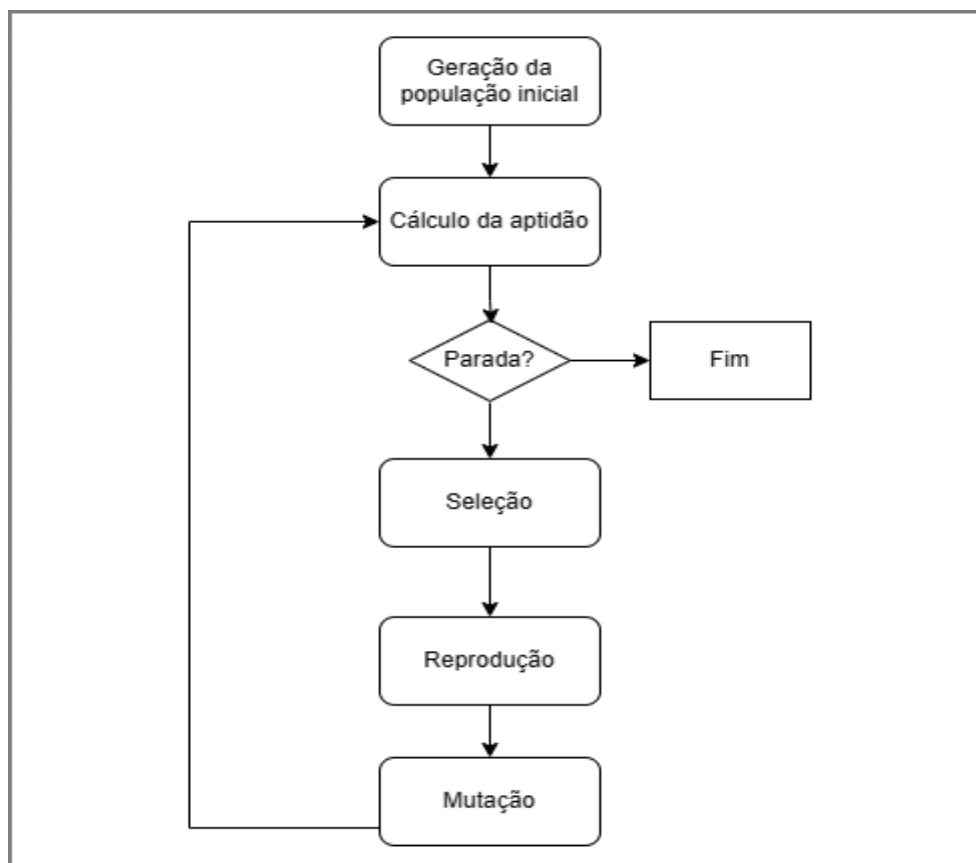


Figura 5 – Fluxo base de um algoritmo genético básico

Na etapa de reprodução, podem ser selecionados os indivíduos com base na aptidão. Esses indivíduos escolhidos geram descendentes por meio de operadores genéticos, onde partes dos genes dos indivíduos selecionados são combinadas para criar novos indivíduos.

Após a criação dos novos indivíduos, é possível que sejam aplicadas mutações. O objetivo dos operadores de mutação é introduzir variações aleatórias, buscando aumentar a variação genética da população. Essa diversidade genética é crucial para evitar a convergência prematura para soluções subótimas, permitindo que o algoritmo explore um amplo espaço de busca em busca de soluções mais eficazes.

Com o tempo, ao longo das iterações do algoritmo genético, espera-se que a população evolua, gerando soluções progressivamente melhores para o problema em questão. Por fim, ao alcançar o critério de parada estabelecido, normalmente número máximo de gerações, é selecionado o indivíduo que representa a melhor solução para o problema.

2.3. Algoritmo Evolucionário com Inspiração Quântica (AEIQ)

Em contraste com um algoritmo genético tradicional, um Algoritmo Evolutivo com Indivíduos Quânticos (AEIQ) requer a representação de indivíduos quânticos por meio de bits quânticos, conhecidos como qubits, sendo sua atualização realizada por portas quânticas, seguindo a definição formulada por Zhang e Gexiang [7].

Um qubit, ao contrário de um bit clássico, possui a capacidade não apenas de representar os estados 0 e 1, mas também de expressar uma sobreposição desses estados, variando conforme a observação realizada.

Para problemas de ordem, conforme em González e outros [5], é proposto que um indivíduo quântico seja representado por uma matriz $n \times m$, onde n é o número de elementos considerados pelo indivíduo quântico e m o número de qubits usados para representar os possíveis estados do indivíduo, podendo representar até 2^m estados distintos. Abaixo, tem-se a generalização proposta para um indivíduo quântico em [4]:

$$Q = \begin{bmatrix} q_{11} & \cdots & q_{1m} \\ \vdots & \ddots & \vdots \\ q_{n1} & \cdots & q_{nm} \end{bmatrix} \text{ onde } q_{ij} \in [0,1] \text{ e } \sum_{j=1}^m q_{ij} = 1, \forall i \in \{1, \dots, n\}$$

Além disso, o processo de geração inicial de um indivíduo quântico difere do processo de um indivíduo clássico. A inicialização consiste na atribuição de probabilidades dos possíveis estados para cada elemento. As únicas restrições são que a probabilidade de um elemento ser selecionado seja menor que 1 e que a soma de todas as probabilidades em uma linha da matriz seja igual a 1. Novamente, seguindo o definido por González e outros [5], para garantir uma maior cobertura do espaço de busca, é recomendado que inicialmente cada elemento do espaço de busca tenha igual probabilidade de ser selecionado.

Além disso, os indivíduos quânticos (diferente de indivíduos clássicos) necessitam de uma etapa adicional chamada de observação. Nessa etapa, o indivíduo quântico, composto por sua matriz de probabilidade, é transformado em um indivíduo clássico. Cada linha da matriz representa um elemento que é selecionado por sorteio, e para evitar repetições, a probabilidade desse elemento ser escolhido novamente é zerada. Assim, as probabilidades das próximas linhas

são atualizadas e o processo é repetido n vezes. Para converter a matriz de valores binários resultante em um indivíduo clássico, é necessário transformá-la em um vetor, indicando o índice da coluna que apresenta um valor diferente de zero em cada linha.

Por fim, outro processo que ocorre diferente em indivíduos quânticos é o processo de atualização. Nesse caso, a atualização é feita utilizando a seguinte fórmula:

$$Q^y = (1 - \varepsilon) * Q^t + \varepsilon * E^t, \varepsilon \in (0, 1]$$

Onde Q^y é o indivíduo quântico atualizado, Q^t é o indivíduo quântico original, E^t matriz resultante da observação do indivíduo quântico Q^t e ε é a taxa de atualização, o valor de ε deve ser ajustado de acordo com as iterações do algoritmo genético, valor menor no início, permitindo um espaço de busca maior e um valor maior nas últimas iterações do algoritmo genético priorizando os melhores indivíduos. Normalmente a atualização é feita apenas em alguns dos indivíduos quânticos que geraram os melhores indivíduos clássicos.

Com os passos mencionados, é viável realizar ações equivalentes a seleção, mutação e avaliação de aptidão a partir de indivíduos quânticos.

2.4. Algoritmo Evolucionário com Inspiração Quântica para a Área de Saúde (AEIQ-AS)

O algoritmo AEIQ-AS [5] tem como base o algoritmo desenvolvido por Silveira em [8].

Na Figura 6 observa-se as etapas de funcionamento do AEIQ-AS. Inicialmente, há a geração da população quântica, composta por indivíduos com o formato descrito no Capítulo 2.3.

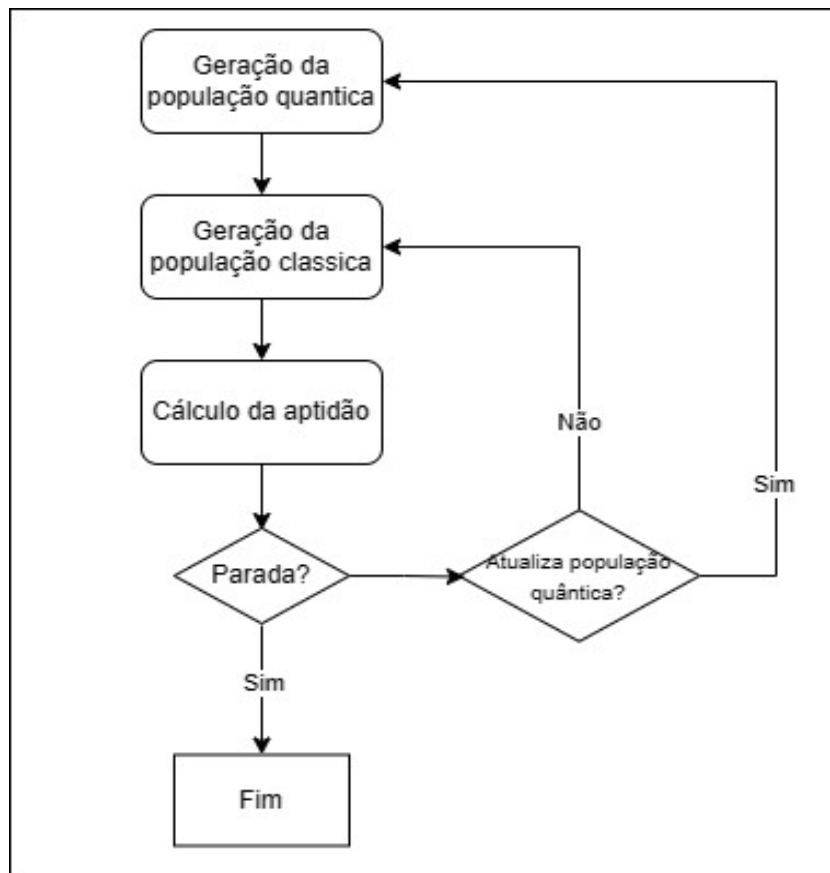


Figura 6 – Fluxo base de um algoritmo genético quântico

Em seguida, ocorre a observação dos indivíduos quânticos para a geração dos indivíduos clássicos, seguida pela avaliação da aptidão desses indivíduos.

Caso a condição de parada não seja atingida, foi adicionada uma condição que apenas executa a atualização de indivíduos quânticos somente após ser atingido um número mínimo de novas gerações clássicas, decidindo se a nova geração de indivíduos clássicos é criada a partir uma população quântica atualizada ou por meio da reprodução dos próprios indivíduos clássicos.

Essa lógica tem como objetivo acelerar o tempo total de execução do algoritmo, visto que, conforme mostrado em González e outros [5], a criação de indivíduos quânticos tende a ser mais custosa computacionalmente que a geração de indivíduos clássicos.

Esse ciclo é repetido até atingir a condição de parada, número máximo de gerações, quando isso acontece o indivíduo clássico que representa a melhor solução é selecionado.

2.5. Servidor

Rocha [6] elaborou um servidor utilizando a linguagem de programação Python [9] em conjunto com o web framework Flask [10]. O objetivo desse servidor é permitir a comunicação, configuração e execução do modelo por meio de uma interface para o usuário. Além disso, também foi realizado o processo de tradução do código original (em Matlab) do modelo para uma versão gerada automaticamente na linguagem C.

A escolha da linguagem Python para a criação do servidor, se deu principalmente por conta da sua fácil integração com a linguagem C, por meio do *CTypes* [11], presente por padrão em Python.

Já o Flask foi escolhido por ser um dos frameworks webs mais comum e permitir um desenvolvimento rápido da integração do modelo com a interface.

2.6. Interface

Já a interface da aplicação foi feita usando *template* HTML usando Jinja [12], presente no Flask. O Jinja é um mecanismo de template flexível que permite a geração dinâmica de conteúdo HTML, facilitando a criação de modelos reutilizáveis.

Na Figura 7 pode-se observar as dependências dos templates HTML desenvolvidos para a interface específica da aplicação, onde cada módulo mostrado na imagem representa um arquivo HTML diferente.

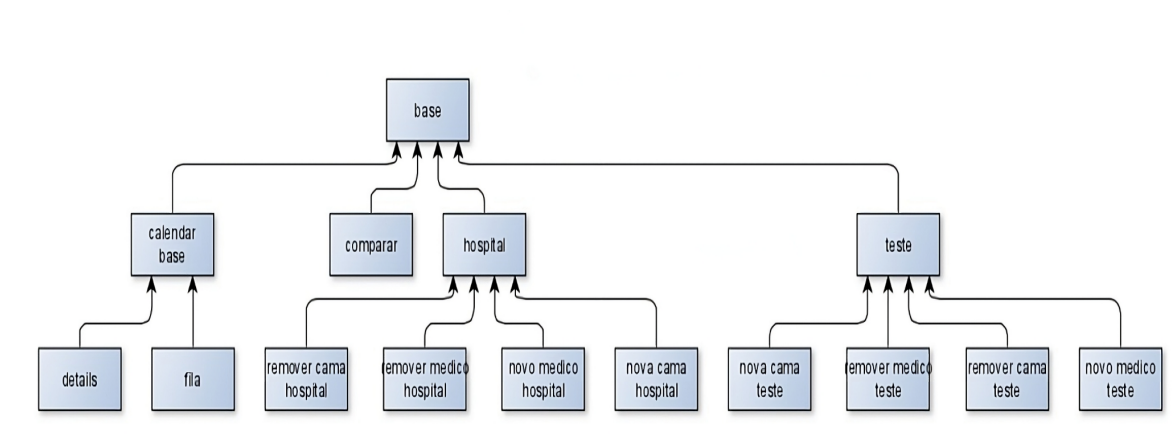


Figura 7 – Tabela de dependência dos templates HTML

3. Objetivos

O objetivo principal desse projeto é a melhorar a separação de responsabilidades do código existente, incluir novas funcionalidades e ajustes na interface da aplicação web para o modelo AEIQ-AS visando a redução das filas de pacientes em procedimentos eletivos. Essa aplicação tem como objetivo permitir a execução e configuração do modelo por um público-alvo não especializado, oferecendo uma maneira simples para os usuários criarem e otimizarem filas para procedimentos eletivos. Além disso, esse trabalho também deve contemplar mudanças no modelo AEIQ-AS para permitir a introdução de novas funcionalidades, assim como também a reestruturação do código existente no servidor e na interface, com o objetivo de facilitar o seu uso e futuros desenvolvimentos.

4. Atividades realizadas

Inicialmente, para obter uma visão mais ampla da estrutura interna do projeto, foi realizado um estudo sobre a estruturação geral de diretórios da aplicação.

Na Figura 8 pode-se visualizar como foi feita a estruturação atual das pastas e arquivos do projeto. Ele é composto por três partes principais. A pasta “C code” contém os arquivos com o código do modelo, juntamente com o arquivo de execução do modelo, utilizado pelo código em Python para realizar a otimização da fila de um hospital. A pasta “Python code” contém os arquivos que servem como ponto de entrada da aplicação, além de também conter os arquivos HTML responsáveis por fornecer a estrutura visual e interativa para a configuração, execução e visualização dos resultados do modelo.

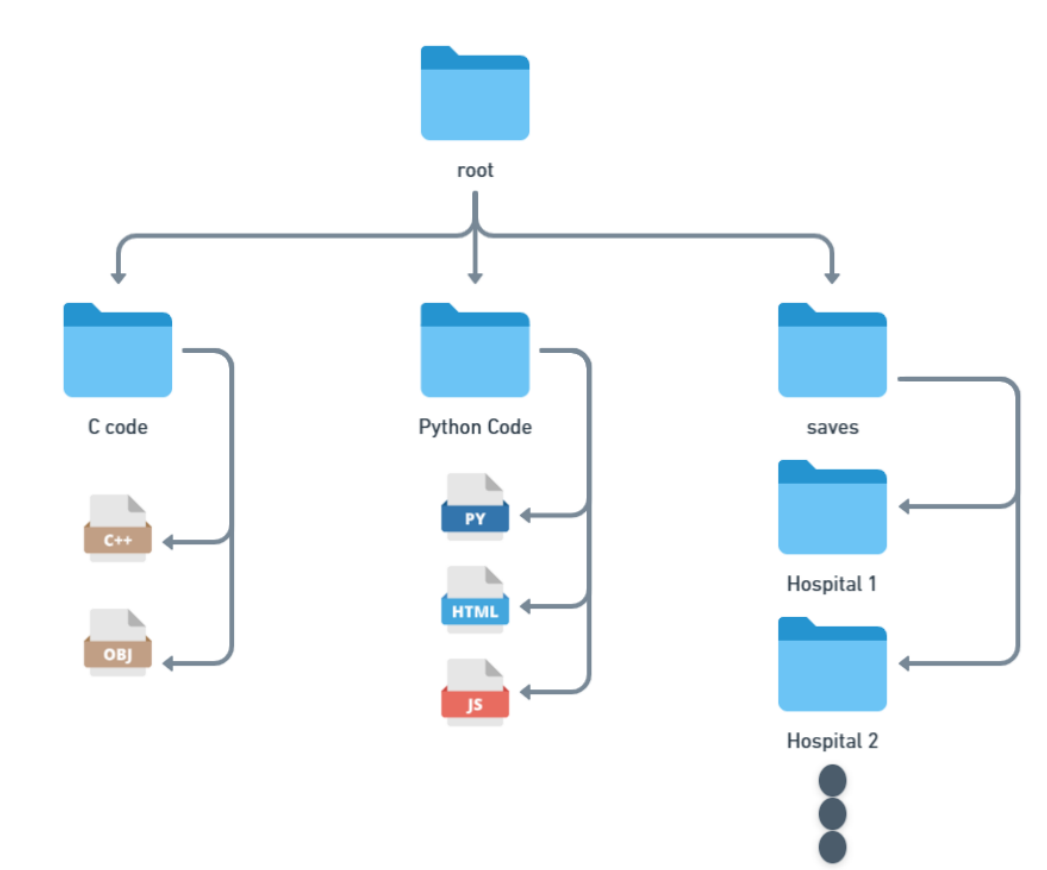


Figura 8 – Estrutura de diretórios do projeto

Já a pasta “saves” é o local onde são criadas as configurações de cada hospital, cada subpasta presente em “saves” representa a configuração de um hospital diferente. Na lista abaixo, é mostrado quais arquivos estão presentes dentro da pasta de um hospital.

- "config.json": contém os valores dos parâmetros do modelo;
- "CP.csv": histórico de pacientes usado para ajustar o modelo;
- "CPO.csv": configuração dos leitos pós-operatórios;
- "CPrO.csv": configuração dos leitos pré-operatórios;
- "CR.csv": configuração das camas de recuperação;
- "Data.csv": contém datas de início e fim do modelo, detalhando dia, mês, ano, hora e o dia total acumulado ao longo dos anos;
- "details.json": contém informações detalhadas sobre cada operação hospitalar, com identificadores, recursos utilizados, título, datas de início e fim, entre outros;
- "Dia.csv": informações diárias, incluindo dia acumulado, dia da semana, início e fim do dia (em horas acumuladas);
- "DispMExD.csv": disponibilidade dos médicos especialistas;

- "events.json": eventos no hospital, menos detalhados que "details.json", utilizados para exibir slots de operações no calendário;
- "MA.csv", "MAAn.csv", "ME.csv": informações sobre médicos assistentes, anestesistas e especialistas, respectivamente;
- "precoFinal.json": preço final da última execução do modelo;
- "precos.json": custo de cada recurso utilizado na última execução do modelo;
- "results.json": valores associados a cada parâmetro do modelo, usados para comparação entre hospitais;
- "RO.csv": informações sobre a operação necessária para cada paciente, incluindo especialidade necessária, complexidade do estado do paciente, entre outros;
- "S.csv": detalhes das salas de cirurgia;
- "schedule.csv": tabela com recursos utilizados a cada hora do calendário gerado após a execução do modelo.

Esses arquivos formam um sistema complexo, onde qualquer mudança nas configurações de um hospital deve ser realizada no arquivo correspondente, mantendo a integridade e a precisão das informações armazenadas para cada hospital.

4.1. Refatoração do Servidor

Quanto à refatoração do servidor, primeiramente é necessário entender a arquitetura inicial do servidor.

Na Figura 9, é possível visualizar a arquitetura inicial do servidor, composta por dois arquivos principais. O primeiro, "model.py", tem a função de encapsular e facilitar a chamada do código do modelo como também salvar os resultados obtidos em seus respectivos arquivos. Ele é usado apenas durante a execução da otimização de filas pelo usuário.

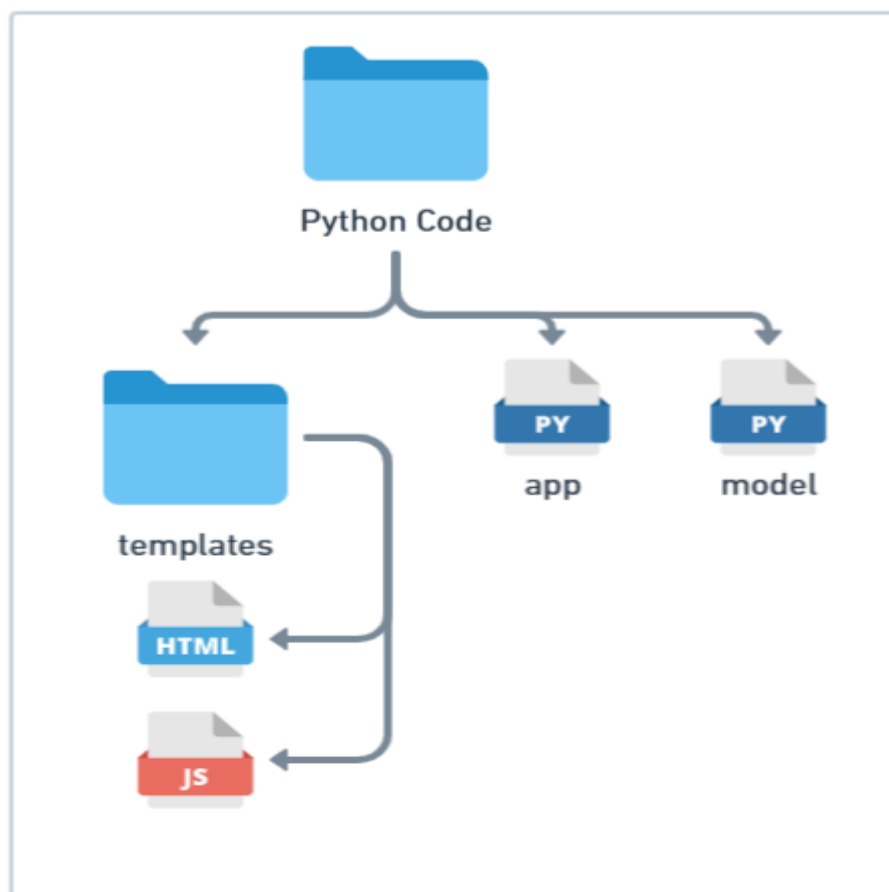


Figura 9 – Estrutura atual do servidor

O "app.py" define as URLs acessíveis ao usuário, estabelecendo a lógica por trás das interações entre o usuário e a aplicação. Neste arquivo são configuradas as rotas para cada funcionalidade da aplicação, determinando como as requisições do cliente são manipuladas e como as respostas são fornecidas, também é responsável pela lógica de armazenamento das configurações do usuário. No entanto, apesar da praticidade inicial dessa abordagem, concentrar toda a lógica da aplicação no arquivo "app.py" pode desencadear alguns problemas à medida que a aplicação evolui, com esse arquivo contendo quase 1000 linhas de códigos abrangendo diversas responsabilidades distintas.

Um dos principais problemas que isso pode causar é afetar a escalabilidade e manutenção da aplicação. Por exemplo, à medida que a aplicação cresce em funcionalidades, o arquivo "app.py" tende a crescer de forma desproporcional. A complexidade excessiva nesse arquivo pode resultar em dificuldades para compreender, modificar e corrigir partes específicas do código, levando a erros mais frequentes e a uma manutenção mais custosa.

Outro problema é a dificuldade de reutilização de código. A falta de uma separação clara de responsabilidades pode comprometer a possibilidade de reutilização de trechos específicos em outras partes da aplicação ou em futuros desenvolvimentos.

Outro ponto crítico é a dependência mútua entre diferentes partes da lógica da aplicação. Isso pode resultar em cenários onde uma pequena modificação em uma parte específica do código pode ter consequências inesperadas e indesejadas em outras partes da aplicação.

Todos esses problemas foram observados e sentidos durante o desenvolvimento das novas funcionalidades da ferramenta, tanto que optou-se por realizar uma refatoração completa da arquitetura atual, visando uma nova arquitetura com maior separação de responsabilidades, com foco em facilitar a compreensão, a manutenção e o desenvolvimento contínuo da aplicação.

Na Figura 10, é possível visualizar a nova arquitetura realizada para o servidor. Ela é baseada na arquitetura recomendada na própria documentação do framework Flask [10]. Essa abordagem tende a permitir uma estrutura mais clara e modularizada, com separação e reutilização das funcionalidades em diferentes partes da aplicação.

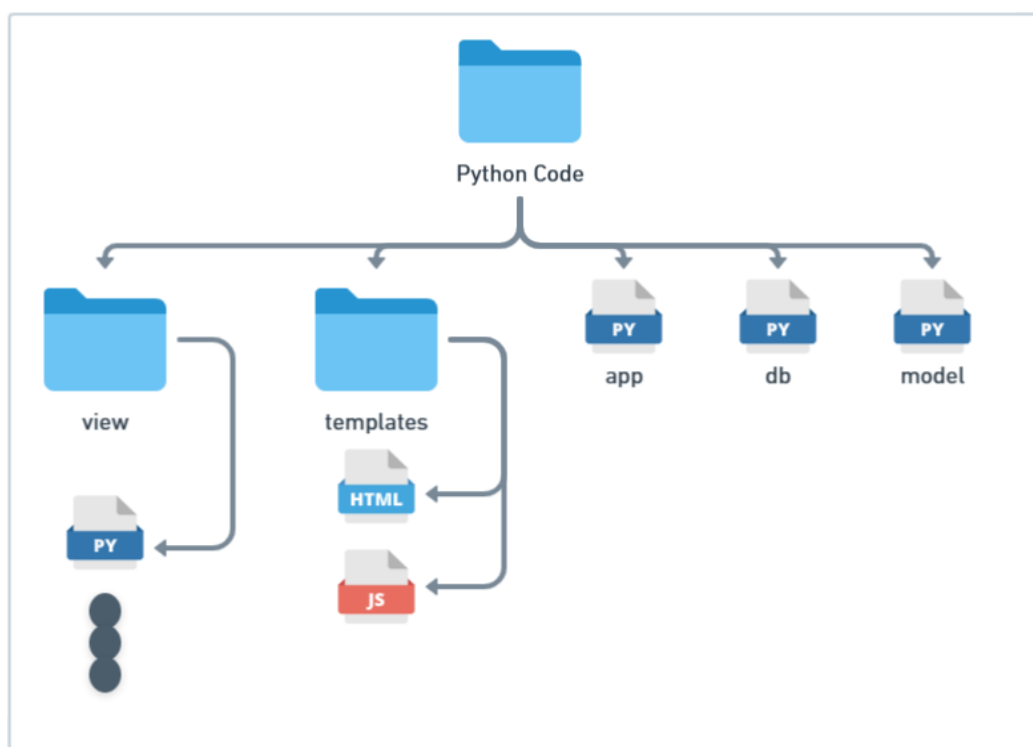


Figura 10 – Estrutura nova servidor

No caso, o arquivo "app.py" passa a ter apenas a responsabilidade de vincular os códigos dos demais arquivos ao framework utilizado. Na pasta "view",

encontram-se os arquivos dedicados a cada tela da aplicação, cada um responsável em responder aos métodos HTTP (como GET, POST, PUT, DELETE) necessários para o funcionamento de cada tela.

Por sua vez, no arquivo “db.py”, estão concentradas as funções que realizam alterações nos dados dos hospitais presentes na pasta “save”. Esse arquivo tem como propósito ser um ponto central para todas as funções que manipulam essas pastas. Assim, em trabalhos futuros, caso haja a adoção de um banco de dados na ferramenta, substituindo um gerenciamento por meio de pastas, bastaria modificar apenas as funções neste arquivo, evitando a necessidade de alterar outras partes do código.

Já o arquivo “model.py” continua com sua função de encapsular a chamada ao código do modelo. Essas mudanças tem por objetivo tornar o código mais legível e devidamente documentado, visando facilitar sua compreensão e facilitar futuras mudanças.

Destaca-se que o principal objetivo dessa mudança foi aumentar a separação de responsabilidades entre diferentes arquivos da aplicação, tornando mais simples a adição de novas funcionalidades e facilitando a compreensão das funcionalidades já existentes.

Além disso, durante a revisão do servidor, várias correções foram implementadas para resolver pequenos erros. Um desses erros ocorria quando o usuário acessava a funcionalidade de testes. Na estrutura anterior, internamente o servidor armazenava em uma variável global qual hospital estava sendo acessado, mas ao acessar a tela de testes, essa variável era inicializada para um valor padrão, perdendo a informação do hospital atual. Assim, caso o usuário acessasse essa tela e fosse, por exemplo, para tela de resultado do modelo, ele estaria vendo um hospital completamente diferente. Outro problema era que a interface não indicava de forma clara qual hospital estava sendo visualizado.

Havia outro problema relacionado à variável global que armazena o hospital acessado. Por se tratar de uma única variável que armazena apenas o nome do hospital, isso impedia que a aplicação fosse usada em múltiplas janelas do navegador. Quando múltiplas abas estavam abertas, ambas sobrescreviavam o valor da variável global, causando a perda de qual hospital está sendo acessado em uma das abas.

Com a revisão do servidor, optou-se por remover a variável que armazena o hospital atual, passando a incluir essa informação na URL do navegador. Essa mudança elimina a necessidade do servidor armazenar esse dado. Com isso, é possível acessar e gerenciar diferentes hospitais simultaneamente, sem o risco

de sobrescrever ou perder informações. Com cada aba ou janela do navegador mantendo sua própria referência, garantindo a independência das sessões e prevenindo conflitos entre elas. Esta abordagem também evita a perda da informação durante a mudança de páginas na ferramenta, uma vez que a única maneira de acessar as informações de um hospital específico é informando o nome do hospital pela URL.

4.2. Refatoração da Interface

Quanto à revisão da interface, as telas existentes, que serão refeitas, podem ser vistas na Figura 7. Os principais objetivos dessa refatoração é preservar as funcionalidades originais, porém, com interfaces mais simples e intuitivas de usar, ao mesmo tempo em que se busca simplificar e tornar o código das interfaces mais conciso e fácil de manter.

Na Figura 11 tem-se os novos arquivos contendo as interfaces da ferramenta, todos os arquivos mostrados na Figura 7, foram condensados nos arquivos referenciados na Figura 11, incluindo os arquivos para novas funcionalidades.

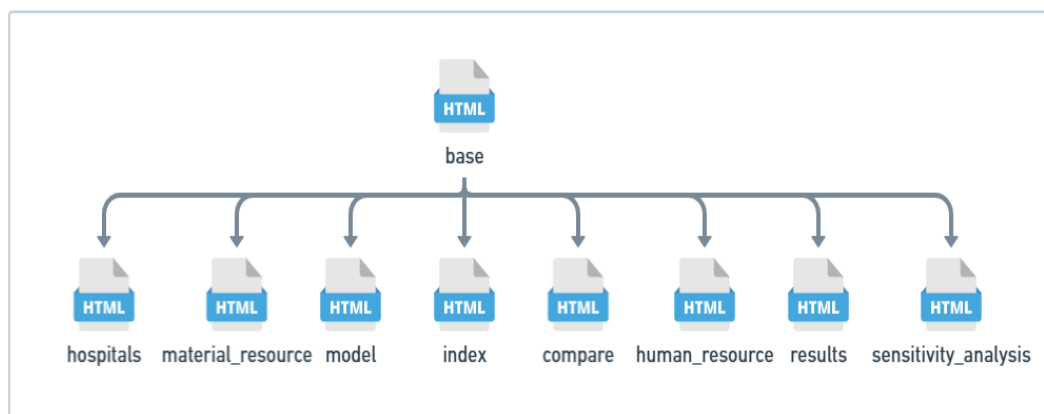


Figura 11 – Estrutura nova da interface

Na Figura 12 pode-se observar a tela gerada pelo arquivo “index.html”, listando todos os hospitais criados, para que o usuário possa acessá-los, junto de algumas informações correspondentes em colunas. Nessa interface, é possível acessar detalhes específicos de cada hospital, acessar a tela de comparação entre hospitais e ainda tem a possibilidade de deletar ou duplicar hospitais específicos.



	Nome	Criação	Data de Execução	
1	hosp (1) com turno	2023-09-30	2023-11-17 10:56:43	
2	hosp-100 - para testes	2023-09-30	2023-11-15 17:42:43	
3	hosp-2000	2023-09-30	-	
4	hosp-300	2023-09-30	2023-11-15 18:47:12	
5	hosp-600	2023-09-30	2023-11-15 19:23:46	
6	test	2023-09-30	-	

Figura 12 – Tela mostrando os hospitais atuais

Já na Figura 13 tem-se a interface referente ao arquivo “hospital.html”, onde são exibidas as possíveis ações dentro de um hospital. Todas essas ações levam para novas telas que serão abordadas mais adiante, além disso agora o usuário tem a opção de alterar o nome do hospital atual.

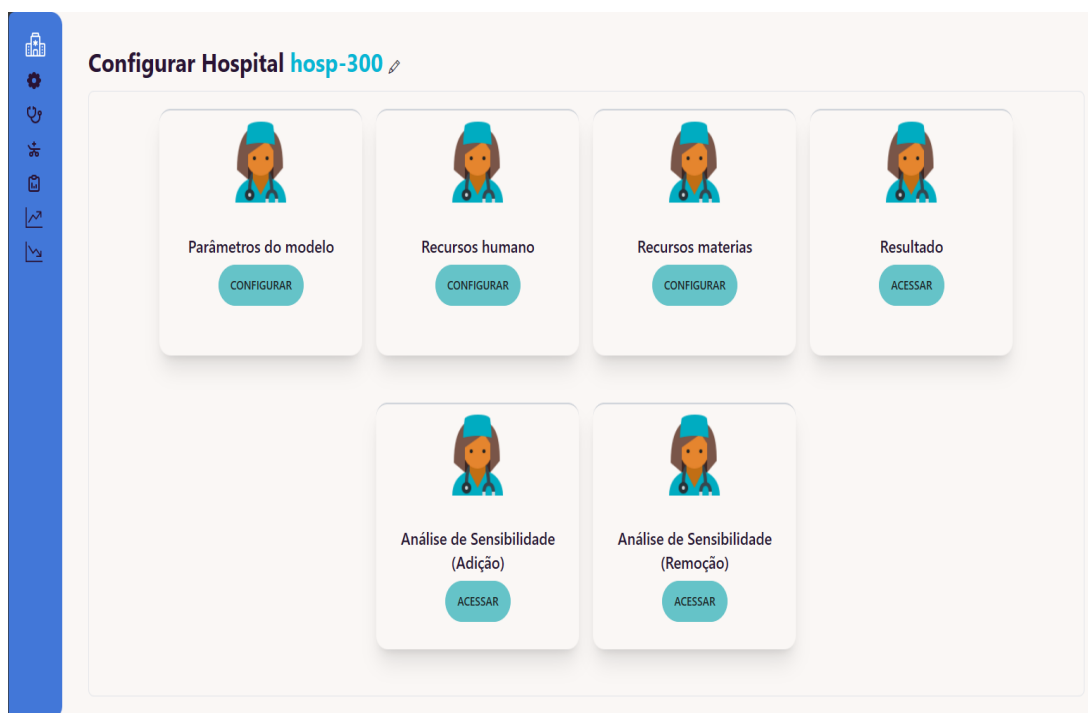


Figura 13 – Tela com as possíveis ações em um hospital

Acrescenta-se que ao selecionar um hospital, as mesmas funcionalidades encontradas na página "hospital.html" estão disponíveis na barra lateral. Cada ícone na barra representa uma dessas operações. Esses ícones são visíveis

somente quando o usuário tem um hospital selecionado, buscando proporcionar um acesso direto e prático às funcionalidades disponíveis para o hospital em questão.

Figura 14 – Tela com os pesos usados pelo modelo

Na Figura 14 tem-se a interface onde são alterados os pesos dos parâmetros utilizados internamente pelo modelo. Os valores k_n são os pesos usados para calcular a pontuação dada pela função de avaliação do modelo, expressa da seguinte forma:

$$f_a = T_{Cir} + k_1 * NFP + k_2 * T_{NFP} + k_3 * NOE_2 + k_4 * NOE_3$$

onde T_{Cir} é o tempo total para conclusão de todas as cirurgias; NFP é o número de cirurgias feitas fora do prazo e k_1 é o seu peso; T_{NFP} é o tempo médio de espera para cirurgias feitas fora do prazo, com k_2 sendo o seu peso; NOE_2 indica o número de operações feitas por especialistas de segundo grau, com k_3 sendo o seu peso; NOE_3 é o número de operações realizadas por especialistas de terceiro grau, com k_4 sendo o seu respectivo peso.

Alterações dos pesos dos parâmetros na fórmula de avaliação do modelo podem impactar significativamente o resultado obtido. Por exemplo, se houver um aumento nos pesos associados ao número de cirurgias feitas fora do prazo (

NFP) ou ao tempo médio de espera para essas cirurgias (T_{NFP}) fará com que o modelo priorize a redução das cirurgias fora do prazo, tentando diminuir o número de suas ocorrências o máximo possível. Por outro lado, uma redução nos valores de NOE_2 e NOE_3 resultará em um aumento no número de procedimentos realizados por médicos especialistas de menor especialidade para essa operação. Este ajuste poderá resultar na diminuição do tempo geral de todas as cirurgias, visto que um maior número de especialistas de menor grau de especialidade estaria disponível para executar procedimentos, agilizando o tempo de execução geral das cirurgias. No entanto, isso também pode impactar nos resultados dos procedimentos realizados, exigindo uma análise cuidadosa antes de realizar mudanças significativas nos pesos dos parâmetros.

Portanto, a adaptação desses pesos deve ser feita levando em consideração não apenas a eficiência operacional, mas também o equilíbrio entre a quantidade e a qualidade dos serviços prestados pelo hospital.

Além disso, os valores relacionados ao custo do uso de recursos materiais e humanos são ajustáveis. Isso permite que, ao final do processo de otimização, seja possível apresentar ao usuário um valor total de uso dos recursos considerados.

A Figura 15 mostra todos os recursos humanos presentes em um hospital específico. A tabela exibe três colunas, uma para o nome, outra para o tipo específico do recurso, com os possíveis valores sendo “Especialista”, “Anestesista” ou “Auxiliar”, e a última coluna indica o ícone para permitir deletar um recurso humano em específico do hospital.

Recursos Humanos de hosp-300 + ADICIONAR MÉDICO

Nome do médico	Tipo	
☐ Medico Especialista 1	Especialista	
☒ Medico Especialista 2	Especialista	
☐ Medico Especialista 3	Especialista	
☐ Medico Especialista 4	Especialista	
☐ Medico Especialista 5	Especialista	
☐ Medico Especialista 6	Especialista	
☐ Medico Especialista 7	Especialista	
☐ Medico Especialista 8	Especialista	

Detalhes do médico Medico Especialista 2

Tipo: Especialista

Lista de especialidades

- Especialidade 1 - Médio
- Especialidade 2 - Alto
- Especialidade 3 - Nenhum
- Especialidade 4 - Baixo
- Especialidade 5 - Médio
- Especialidade 6 - Médio
- Especialidade 7 - Nenhum
- Especialidade 8 - Nenhum
- Especialidade 9 - Nenhum
- Especialidade 10 - Nenhum
- Especialidade 11 - Nenhum
- Especialidade 12 - Médio
- Especialidade 13 - Baixo

Figura 15 – Tela com a lista de recursos humanos de um hospital específico.

Ao selecionar uma linha da tabela, um painel com detalhes do recurso escolhido é exibido, sendo listado ao lado, as especialidades e seus respectivos níveis (ver Figura 16), e caso o médico selecionado seja um especialista também será mostrado as disponibilidades dele (ver Figura 17).

The interface displays a table of medical specialists and a panel for adding a new resource. The table has columns for 'Nome do médico' and 'Tipo'. The panel on the right is titled 'Adicionando' and includes a dropdown for 'Selecione o tipo do médico', a text input for 'Nome do médico', and a grid of 15 speciality options, each with a 'Nenhum' dropdown.

Nome do médico	Tipo
☐ Medico Especialista 1	Especialista
☐ Medico Especialista 2	Especialista
☐ Medico Especialista 3	Especialista
☐ Medico Especialista 4	Especialista
☐ Medico Especialista 5	Especialista
☐ Medico Especialista 6	Especialista
☐ Medico Especialista 7	Especialista
☐ Medico Especialista 8	Especialista

Adicionando Seleção o tipo do médico

Nome do médico

Nome do médico

Especialidades

Especialidade 1	Nenhum	Especialidade 2	Nenhum
Especialidade 3	Nenhum	Especialidade 4	Nenhum
Especialidade 5	Nenhum	Especialidade 6	Nenhum
Especialidade 7	Nenhum	Especialidade 8	Nenhum
Especialidade 9	Nenhum	Especialidade 10	Nenhum
Especialidade 11	Nenhum	Especialidade 12	Nenhum
Especialidade 13	Nenhum	Especialidade 14	Nenhum
Especialidade 15	Nenhum		

Figura 16 – Tela com a lista de recursos e painel de criação de recursos.

The interface displays the same table of medical specialists and a panel for adding a new resource. The panel on the right is titled 'Adicionando' and includes a dropdown for 'Selecione o tipo do médico', a text input for 'Nome do médico', and a grid of 15 speciality options, each with a 'Nenhum' dropdown. Below the speciality options, there is a section for 'Disponibilidades' with radio buttons for each day of the week and time slot (Manhã, Tarde, Noite). The 'Terça-feira Tarde' and 'Quinta-feira Manhã' options are selected. A blue 'ADICIONAR' button is at the bottom.

Nome do médico	Tipo
☐ Medico Especialista 1	Especialista
☐ Medico Especialista 2	Especialista
☐ Medico Especialista 3	Especialista
☐ Medico Especialista 4	Especialista
☐ Medico Especialista 5	Especialista
☐ Medico Especialista 6	Especialista
☐ Medico Especialista 7	Especialista
☐ Medico Especialista 8	Especialista

Adicionando Seleção o tipo do médico

Nome do médico

Nome do médico

Especialidades

Especialidade 5	Nenhum	Especialidade 6	Nenhum
Especialidade 7	Nenhum	Especialidade 8	Nenhum
Especialidade 9	Nenhum	Especialidade 10	Nenhum
Especialidade 11	Nenhum	Especialidade 12	Nenhum
Especialidade 13	Nenhum	Especialidade 14	Nenhum
Especialidade 15	Nenhum		

Disponibilidades

Segunda-feira ☐ Manhã ☐ Tarde ☐ Noite

Terça-feira ☐ Manhã ☒ Tarde ☐ Noite

Quarta-feira ☐ Manhã ☐ Tarde ☐ Noite

Quinta-feira ☒ Manhã ☐ Tarde ☐ Noite

Sexta-feira ☐ Manhã ☐ Tarde ☐ Noite

Sábado ☐ Manhã ☐ Tarde ☐ Noite

ADICIONAR

Figura 17 – Tela com as opções de turno para médico de tipo especialista.

Além disso, ao clicar no botão de criação de um novo médico, é exibido um painel para inserir os parâmetros do profissional. Todos os profissionais possuem campos de especialidade. Caso seja inserido um médico do tipo especialista, são mostrados, além das especialidades, as opções de turnos do médico, conforme exibido na Figura 17.

Na Figura 18 é exibido a interface de recursos materiais que opera de forma semelhante à tela de recursos humanos. A distinção está nos tipos de recursos, agora sendo “Cama Pré-Operatória”, “Cama Pós-Operatória”, “Cama de repouso” e “Sala de Operação”, e a especialidade não é mais medida em nível, mas sim utilizando booleanos, que indicam se é possível utilizar o recursos em operações daquela especialidade específica, conforme exibido na Figura 19.

Recursos Materiais de hosp-300 + ADICIONAR RECURSO

Nome do recurso	Tipo	Ações
☐ Cama Pre Operatoria 1	pre	
☒ Cama Pre Operatoria 2	pre	
☐ Cama Pre Operatoria 3	pre	
☐ Cama Pre Operatoria 4	pre	
☐ Cama Pre Operatoria 5	pre	
☐ Cama Pre Operatoria 6	pre	
☐ Cama Pre Operatoria 7	pre	
☐ Cama Pre Operatoria 8	pre	

Especialidade 1 - Sim
Especialidade 2 - Sim
Especialidade 3 - Sim
Especialidade 4 - Sim
Especialidade 5 - Não
Especialidade 6 - Sim
Especialidade 7 - Sim
Especialidade 8 - Sim
Especialidade 9 - Sim
Especialidade 10 - Sim
Especialidade 11 - Sim
Especialidade 12 - Sim
Especialidade 13 - Sim
Especialidade 14 - Sim
Especialidade 15 - Sim

Figura 18 – Tela com recursos materiais.

Recursos Materiais de hosp-300 + ADICIONAR RECURSO

Nome do recurso	Tipo	Ações
☐ Cama Pre Operatoria 1	pre	
☒ Cama Pre Operatoria 2	pre	
☐ Cama Pre Operatoria 3	pre	
☐ Cama Pre Operatoria 4	pre	
☐ Cama Pre Operatoria 5	pre	
☐ Cama Pre Operatoria 6	pre	
☐ Cama Pre Operatoria 7	pre	
☐ Cama Pre Operatoria 8	pre	

Adicionando Selecione o tipo o recurso.

Nome do recurso

Nome do recurso

Especialidades

Especialidade 1	☒	Especialidade 2	☒
Especialidade 3	☒	Especialidade 4	☒
Especialidade 5	☒	Especialidade 6	☒
Especialidade 7	☒	Especialidade 8	☒
Especialidade 9	☒	Especialidade 10	☒
Especialidade 11	☒	Especialidade 12	☒
Especialidade 13	☒	Especialidade 14	☒
Especialidade 15	☒		

ADICIONAR

Figura 19 – Tela com painel de adição de recursos materiais.

Na Figura 20 é exibido o que é visível quando o usuário acessa a tela de resultados sem ter executado o modelo anteriormente.

Fila de hosp - com turno 1 Filtro: Todos os recursos

Execute o modelo para visualizar o resultado.

EXECUTAR

Figura 20 – Página de execução do modelo.

Na Figura 21, após a execução do modelo em uma fila, é exibido o calendário das operações. É possível configurar o calendário para mostrar as operações no formato mensal ou diário, sendo o padrão a visualização de todas as operações em um mês. Além disso, também há um campo de filtro onde é possível selecionar qual tipo de eventos serão mostrados no calendário, por padrão todos os eventos são mostrados.

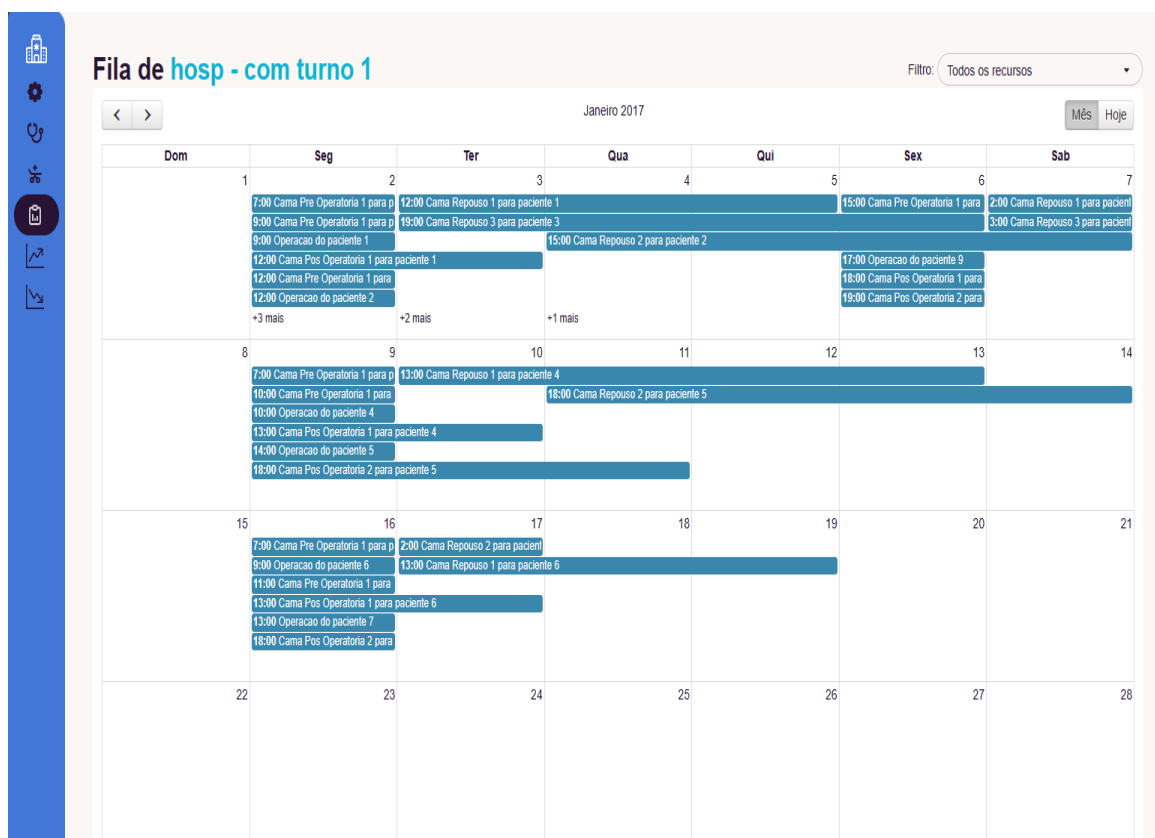


Figura 21 – Página de execução com resultado da execução.

A Figura 22 apresenta o gráfico quando é selecionado o modo diário, oferecendo uma visão mais geral das operações marcadas para aquele dia, facilitando a identificação dos horários de início e término de cada operação. Além disso, foram incluídos mais detalhes nos títulos mostrados no calendário, como informações sobre os tipos de recursos utilizados junto da identificação do paciente associado a cada operação.

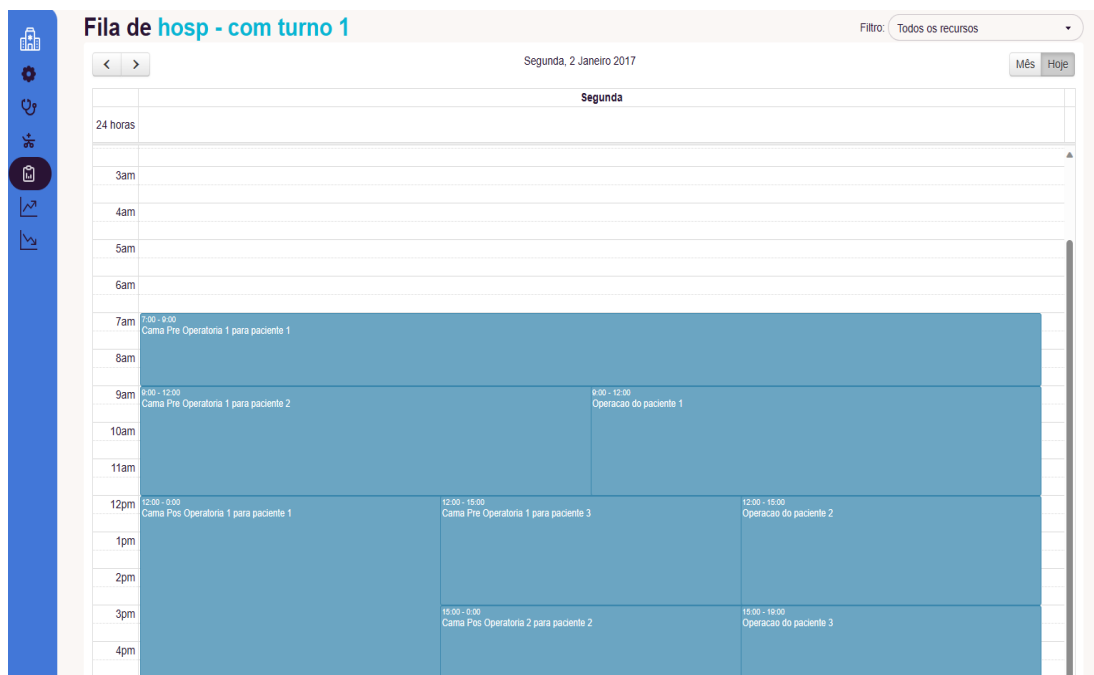


Figura 22 – Página de execução com calendário no modo diário.

Na Figura 23 são apresentados os detalhes de uma operação específica, acessíveis ao clicar em um evento no calendário. Esses detalhes incluem informações sobre o início e término da operação, sua duração total e os recursos utilizados durante a operação.

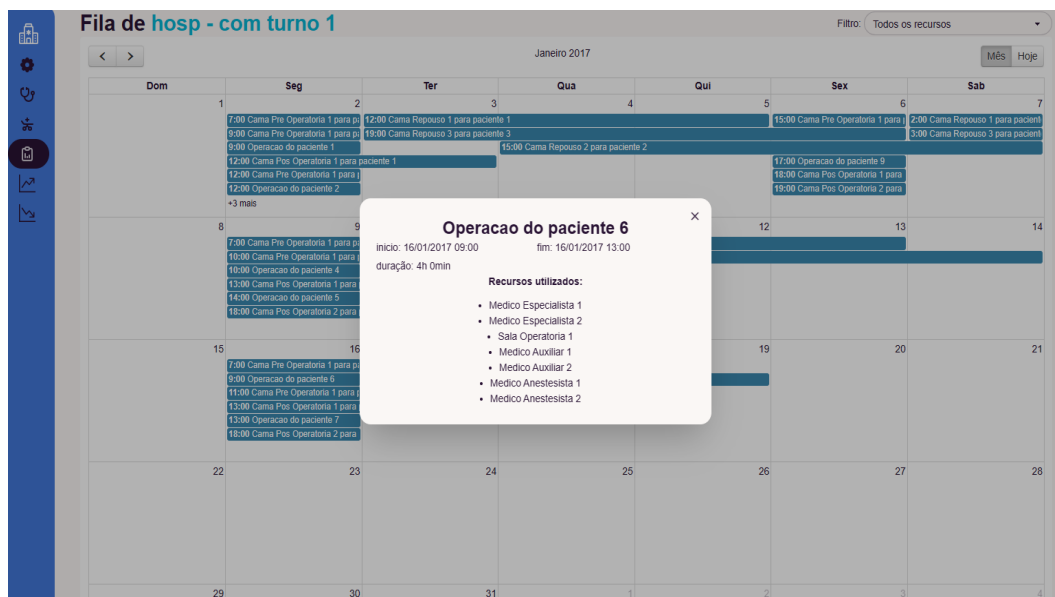


Figura 23 – Página de execução mostrando os detalhes de uma operação

Na Figura 24 é exibido a página onde é feita a comparação entre resultados dos modelos, nela é possível comparar os resultados obtidos a partir da seleção de duas configurações de recursos em hospitais, exibindo para cada hospital os valores usados para calcular sua pontuação, baseada na função de

avaliação. Mais detalhes da função de avaliação podem ser vistos ao posicionar o cursor do mouse sobre o valor da pontuação, como mostrado na Figura 25.

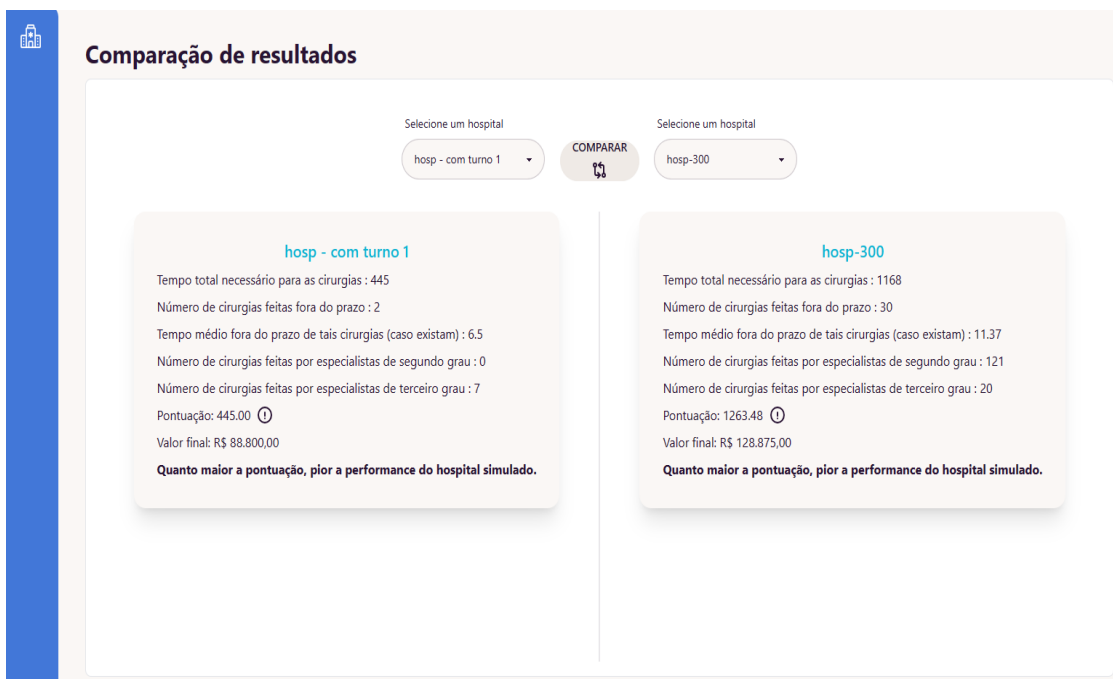


Figura 24 – Página de comparação entre resultados

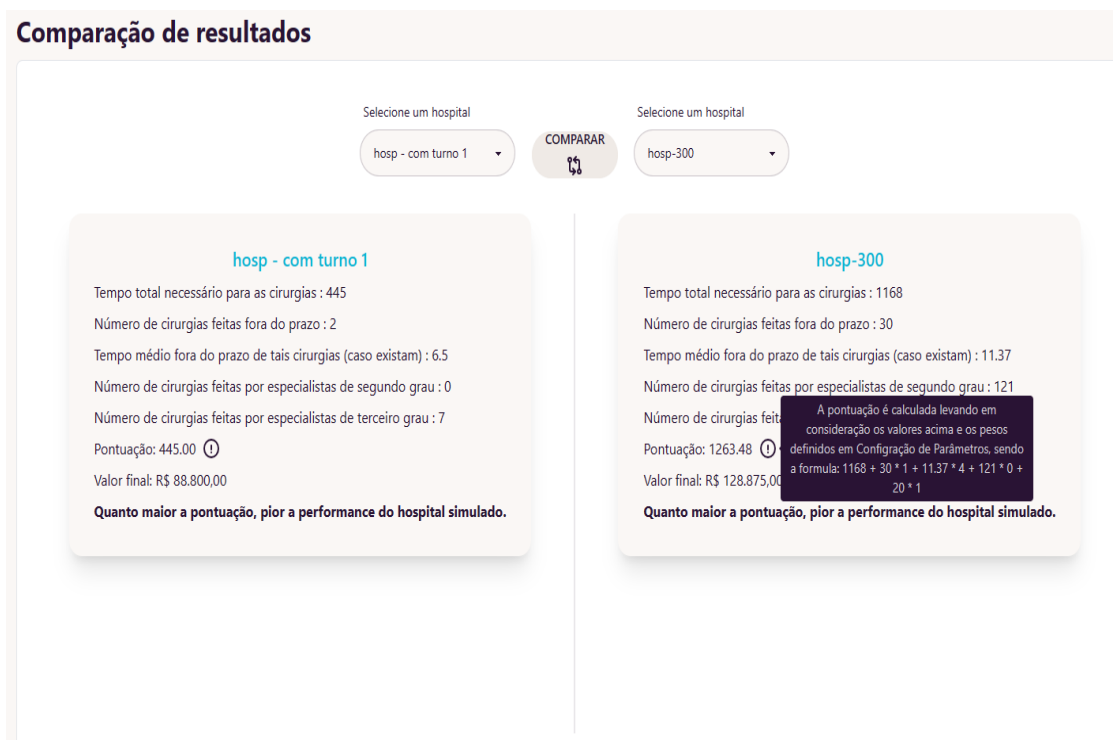


Figura 25 – Página de comparação com detalhes da pontuação

4.3. Análise de sensibilidade

De acordo com a definição apresentada por Tsonis e colegas [13], a análise de sensibilidade (do inglês sensitivity analysis - SA) é um estudo que determina o impacto de uma ou mais variáveis sobre um sistema. Esses sistemas normalmente são modelos matemáticos codificados por meio de software computacional. Essencialmente, ele busca identificar as variáveis mais influentes em um sistema e compreender como mudanças nessas variáveis impactam os resultados de modelo.

No caso do modelo AEIQ-AS, a análise de sensibilidade pode ser utilizada para avaliar o impacto dos recursos materiais e humanos na pontuação dos resultados. Com essa finalidade, foram desenvolvidas duas abordagens distintas de SA. Na primeira, partindo de um caso base, cada recurso específico é inserido individualmente no hospital. Para isso, são criadas múltiplas instâncias do caso base, onde cada cópia recebe a adição de apenas um recurso específico. A Figura 26 ilustra com melhor detalhes esse processo.

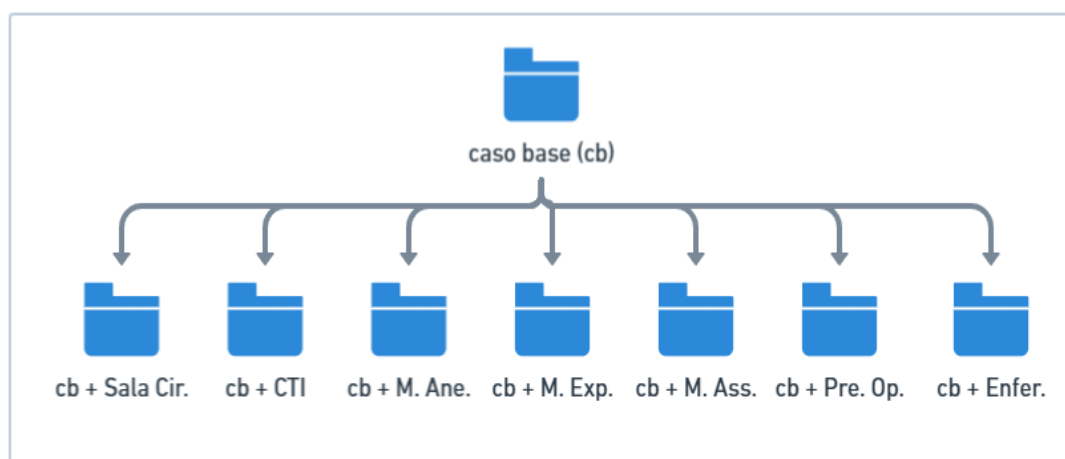


Figura 26 – Diagrama com a estrutura de uma análise de sensibilidade

Para testar a análise de sensibilidade, foi criado um hospital com os recursos presentes na Tabela 1.

Descrição	Quantidade
Cirurgias	30
Leitos Pré-operatórios	10
Salas de Cirurgias	10
Médicos Especialistas	3
Assistente	10

Descrição	Quantidade
Anestesistas	10
Leitos de CTI	25
Leitos de Enfermagem	9

Tabela 1 - Recursos do hospital de teste

Além disso, para direcionar o foco na otimização do tempo total das cirurgias, foi dado peso zero a todos os parâmetros da função de avaliação. Isso resultou na seguinte função de avaliação:

$$f_a = T_{Cir}$$

Na Figura 27 tem-se o resultado da execução da análise de sensibilidade para o caso de adição de recursos. Nessa figura, é apresentado um gráfico de barras que, nesse caso, compara a diferença entre a pontuação de cada caso em relação à pontuação do caso base. Como o objetivo é minimizar a pontuação, os casos com valores negativos tiveram pontuação inferior ao caso base, e portanto um resultado melhor que o caso base. Apenas a adição de um médico especialista gerou uma fila com pontuação inferior ao caso base. Como pode-se ver na Tabela 1, considerando a quantidade de cada recurso, torna-se evidente que o médico especialista é a possível restrição nessa configuração, e o resultado da análise de sensibilidade corrobora com essa suposição.

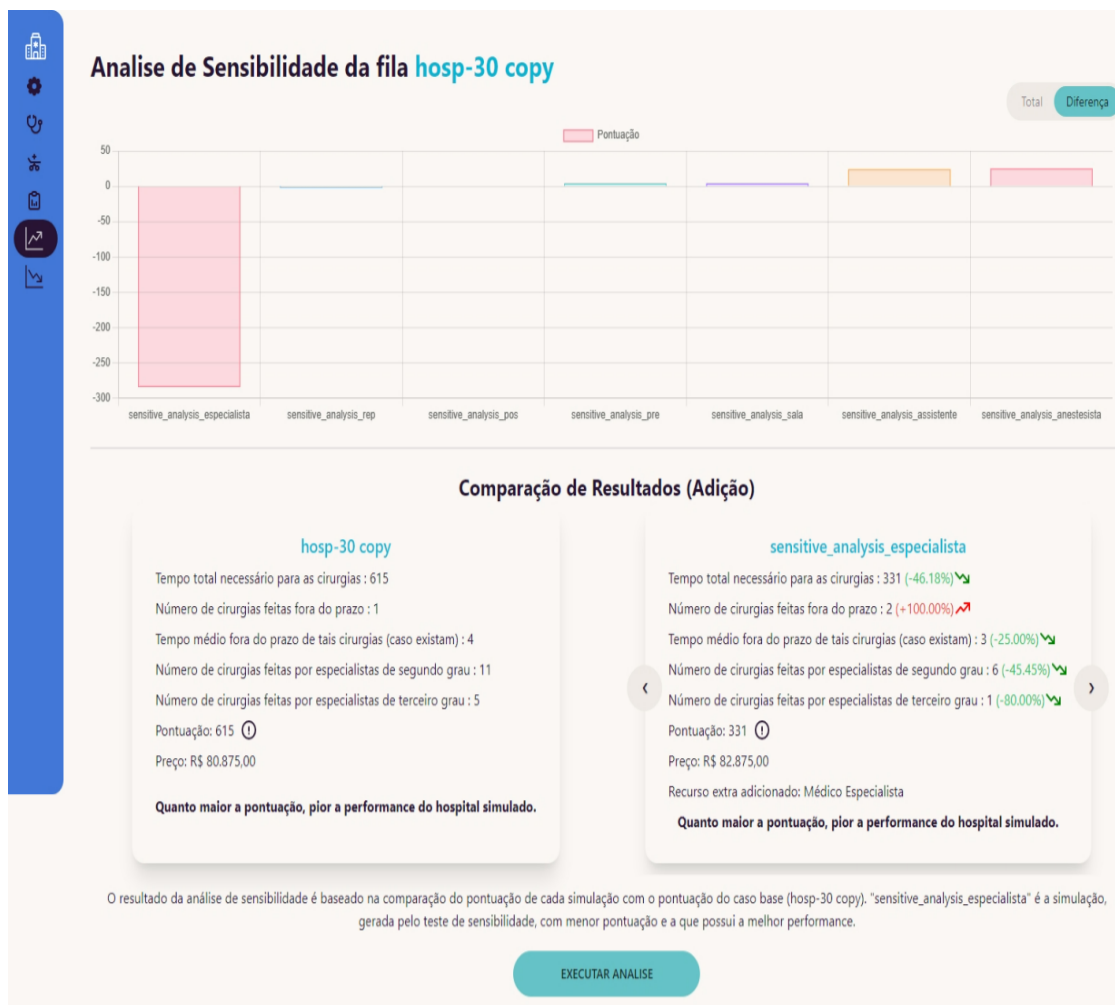


Figura 27 – Tela de resultado da execução uma análise de sensibilidade de adição

Na Figura 28 encontra-se o resultado de uma análise de sensibilidade de remoção, na qual, ao invés de adicionar um recurso, como na análise anterior, um recurso específico é removido para cada caso. O resultado obtido foi coerente com o resultado da análise de adição, no caso, ao remover um dos médicos especialistas, houve um considerável aumento na pontuação do hospital. Isso reforça a indicação de que o gargalo desse hospital reside na quantidade de médicos especialistas, enquanto a pontuação dos outros casos tiveram pouca ou nenhuma mudança em relação ao caso base.

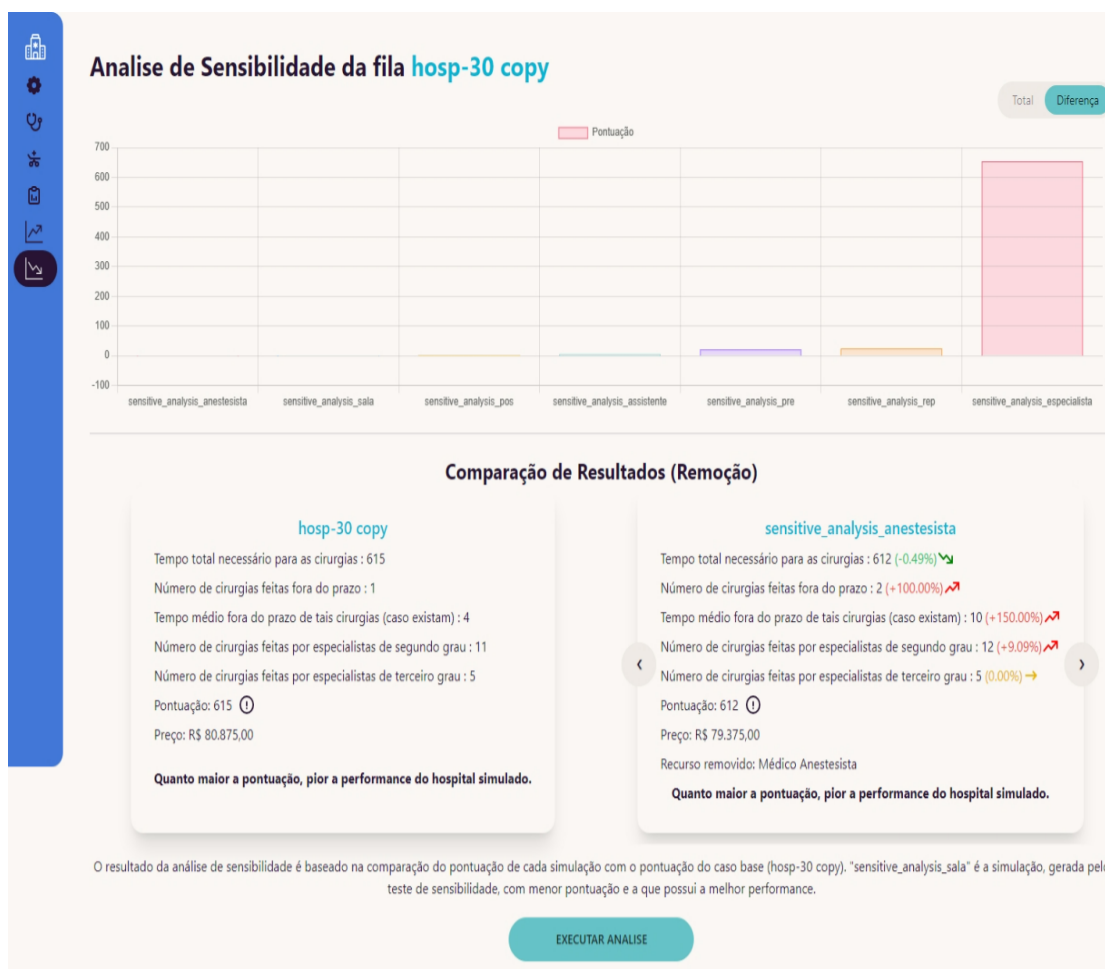


Figura 28 – Tela de resultado da execução uma análise de sensibilidade de remoção

Além disso, na tela de análise de sensibilidade, o gráfico de barras possui dois modos: "Total" e "Diferença". No modo "Total", são exibidos os valores reais das pontuações de cada análise, incluindo a barra do caso base. Entretanto, em casos de diferenças muito pequenas entre as barras, pode ser difícil visualizar essa variação. O modo "Diferença" é mais recomendado para uma melhor visualização dos resultados, já que nesse modo a barra do caso base é omitida, e cada pontuação resultante da análise é apresentada como a diferença em relação à pontuação do caso base. Isso torna a visualização mais clara, como exemplificado nas Figuras 27 e 28.

Outra forma disponível para visualizar os resultados é através dos blocos individuais de cada caso da análise, com acesso disponível no canto inferior direito da janela. Esses blocos exibem o nome de cada caso gerado pela análise de sensibilidade, juntamente do valor individual de cada variável considerada pela função de avaliação. Cada valor é comparado com o seu equivalente no

caso base, apresentando indicadores visuais que sinalizam melhora ou piora em relação ao seu equivalente. É importante ressaltar que, devido ao problema ser de minimização, uma melhoria no resultado significa uma diminuição do valor em comparação com o caso base correspondente. Para facilitar a compreensão, também foi incluído um texto indicativo do recurso adicional inserido em cada caso do SA, visto que o nome de cada caso é um nome padrão.

4.4. Introdução de turnos no AEIQ-AS

Para incorporar os turnos dos médicos especialistas ao modelo, foi necessário alterar o seu funcionamento. Inicialmente, o modelo considerava somente dias completos, nos quais os médicos trabalhavam durante todas as horas úteis do dia ou não trabalhavam naquele dia. Agora, com a implementação dos turnos, os médicos podem ter jornadas diferentes ao longo do dia, para isso foi necessário alterações nos códigos de dois arquivos: "funcionDia.c" e "funcionCPrO.c".

O modelo considera que cirurgias podem acontecer somente durante o horário de trabalho dos médicos especialistas. Nele é estabelecido que de segunda a sexta-feira, o horário de trabalho é das 8h às 20h, no sábado, das 8h às 16h, e não há disponibilidade para cirurgias durante os domingos. Para ajustar isso, optou-se por dividir o dia em até três turnos distintos: manhã, das 8h às 11h59min, tarde, das 12h às 15h59min, e noite, das 16h às 19h59min.

No entanto, essa atualização exigiu uma reformulação no código do modelo. Como o código anterior em MATLAB foi traduzido para seu equivalente em C, surgiu um desafio considerável para essa tarefa. Isso ocorre pois 100 linhas de código em MATLAB podem se tornar milhares de linhas em seu equivalente em C. Outro ponto, é que a tradução automática do código não tem preocupação com clareza e legibilidade, usando frequentemente nomes de variáveis curtos, às vezes compostos por apenas três caracteres. Essa conversão também inclui muitas otimizações internas que teoricamente melhoram o tempo de execução do código, mas que adiciona complexidade ao entendimento do código. Para compreender o código em C, a estratégia mais eficaz tem sido consultar os trechos comentados do código MATLAB original, já que a tradução preserva essas partes de forma comentada.

A primeira mudança foi no modo em como a informação é passada para o modelo. Anteriormente, as informações dos dias de trabalho dos médicos especialistas eram representadas por uma matriz $6 \times n$, onde n é o número de

médicos especialistas inseridos no hospital, e 6 o número de dias válidos para realizar operações, de segunda a sábado. Com a introdução dos turnos, a matriz de disponibilidade foi expandida para $18 \times n$, onde cada conjunto de três linhas representa os turnos de um único dia da semana, de manhã à noite

A Figura 29 mostra uma das mudanças necessárias para a leitura do formato atualizado de dados na matriz de disponibilidade, no código DispME. Nela também é possível notar como poucas linhas de código em MATLAB podem se transformar em uma quantidade considerável de linhas no código convertido, além da presença de algumas otimizações que são feitas ao gerar o código.

```
/* 'funcionDia:34' switch Dia(contDia,2) */
switch (Dia->data[1 + ((*contDia - 1) << 2)])
{
case 2:
    /* 'funcionDia:35' case 2 */
    /* 'funcionDia:36' DispME=int32(DispMExD(1,:)); */
    loop_ub = DispMExD->size[1];
    i49 = DispME->size[0] * DispME->size[1];
    DispME->size[1] = loop_ub;
    DispME->size[0] = 1;
    emxEnsureCapacity_int32_T(DispME, i49);
    for (i49 = 0; i49 < loop_ub; i49++)
    {
        const manha = i49 + DispMExD->size[1] * 0;
        const tarde = i49 + DispMExD->size[1] * 1;
        const noite = i49 + DispMExD->size[1] * 2;
        DispME->data[i49] = (DispMExD->data[manha] << 0) +
            (DispMExD->data[tarde] << 1) +
            (DispMExD->data[noite] << 2);
    }
    break;
}
```

Figura 29 – Exemplo do código alterado em “funcionDia.c”

Com a atualização do código considerando os três turnos por dia, é necessário ajustar o código em “funcionCPro.c”, que trata do agendamento do uso do recurso, para considerar o turno de trabalho dos médicos.

Na Figura 30, o trecho de código apresentado foi adicionado para que o agendamento dos recursos, que requerem médico especialista, considerem o turno de trabalho do médico. Para auxiliar futuros trabalhos, todo o código foi adicionado com comentários explicativos sobre sua função. Espera-se que esses comentários facilitem futuras manutenções no código.

```

// Calculate the start of the day in accumulated hours
const int day_start_acc_hours = Dia->data[2 + ((*contDia - 1) << 2)];

// Calculate the possible start of the pre-operation bed
const int possible_start_preOperationBed = Dia->data[minP_tmp] + tempUltPosRecXDia->data[temCPro - 1];

// Calculate the difference between the possible start of the pre-operation bed and the start of the day
const int diff = possible_start_preOperationBed - day_start_acc_hours;

// Get the current weekday
const int week_day = Dia->data[1 + ((*contDia - 1) << 2)];

// Initialize time_skip to 0
int time_skip = 0;

// Check if the possible start of the pre-operation bed is in the morning, afternoon, or night shift
const bool is_cpro_in_morning_shift = diff >= 0 && diff < 4;
const bool is_cpro_in_afternoon_shift = diff >= 4 && diff < 8;
const bool is_cpro_in_night_shift = diff >= 8 && diff < 12;

// Initialize variables to store if there is a medic available in the morning, afternoon, or night shift
bool has_medic_in_morning_shift = false;
bool has_medic_in_afternoon_shift = false;
bool has_medic_in_night_shift = false;

// Loop over all possibiles medics, a medic is available if it has the corresponding bit set to 1
// for the actual week day
for (int i = 0; i < allE->size[1]; i++)
{

```

Figura 30 – Exemplo do código alterado em “funcionCPro.c”

Na Figura 31 é mostrado um exemplo funcional de operações cirúrgicas que levam em consideração o turno do paciente. No caso da Figura 31, o calendário foi filtrado para mostrar apenas eventos cirúrgicos. Nesse hospital, foram colocados apenas dois médicos, no qual um deles trabalha os três turnos da segunda e o outro trabalha apenas no turno da noite da sexta-feira.

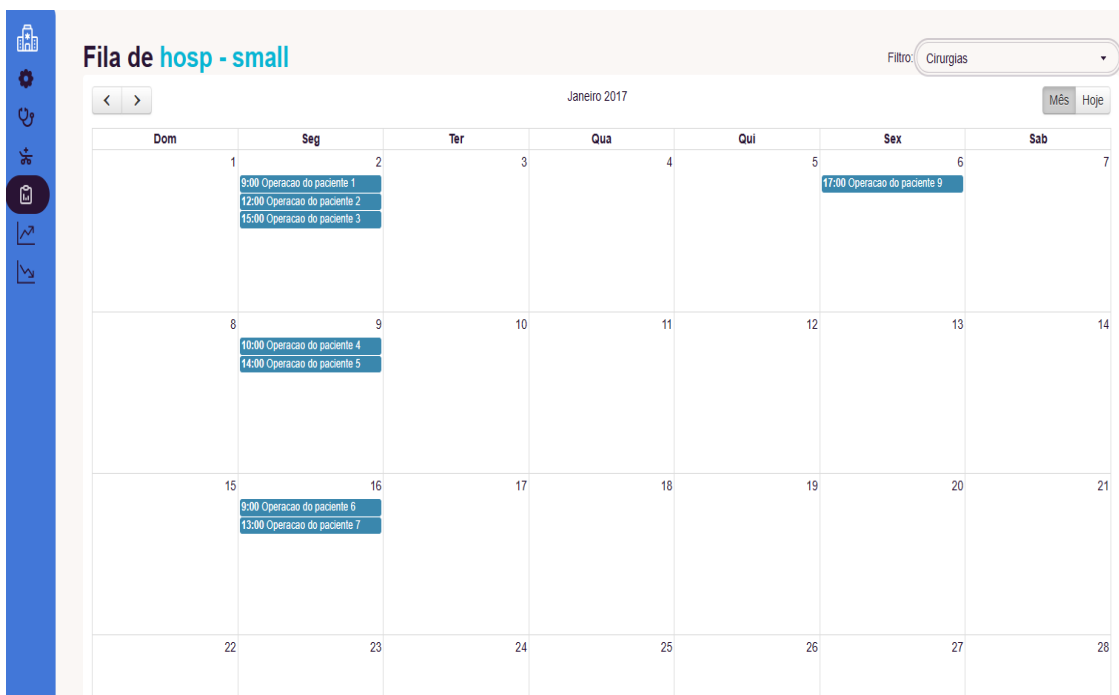


Figura 31 – Exemplo execução com turnos, filtrado por cirurgia.

5. Considerações finais e trabalhos futuros

O principal objetivo deste trabalho foi a correção de diversos problemas identificados na versão anterior, extensão da aplicação com novas funcionalidades, e o aprimoramento da documentação sobre o funcionamento interno da aplicação, incluindo o mapeamento das funções de cada arquivo. Ao longo do processo de desenvolvimento, ficou evidente a necessidade de refatorar uma parte significativa da base de código da aplicação, um processo detalhado no Capítulo 4.

Acredito que esse esforço de refatoração teve um impacto positivo, já que a implementação das novas funcionalidades se tornou relativamente mais simples em comparação com tentativas anteriores à refatoração. No geral, a nova arquitetura foi percebida como mais simples e fácil de ser mantida em comparação com a anterior.

Durante a realização do projeto, enfrentou-se dificuldades principalmente relacionadas à compreensão do código original, que continha funções com nomes que não refletiam a lógica implementada neles ou o uso de variáveis globais que eram utilizadas em lugares que não deveriam ser.

Uma crítica autocrítica que destaco neste trabalho e que serve como aprendizado para projetos futuros é o uso excessivo do Jinja em determinados trechos do código de interface. O Jinja é um mecanismo de template usado no Flask, para geração de conteúdo dinâmico em páginas web. Contudo, o seu uso acaba por adicionar uma complexidade desnecessária ao código, sendo necessário recorrer com frequência a documentação do Jinja. Por esse motivo, optou-se por minimizar seu uso no estágio final do desenvolvimento, tornando o código das páginas web mais recentes mais claro e legível do que as versões anteriores. Entretanto, o trabalho de refatorar as partes onde o Jinja é muito utilizado poderia ser incluído em trabalhos futuros.

5.1. Trabalhos Futuros

Neste capítulo serão abordados alguns tópicos que podem ser abordados em trabalhos futuros que considero que tenham um bom valor para a aplicação.

5.1.1. Banco de dados

Atualmente, a aplicação utiliza um sistema de pasta para gerenciar o estado dos hospitais, contudo essa forma de gerenciar apresenta suas limitações. Uma delas é a escalabilidade da aplicação, a medida que novas

funcionalidades sejam implementadas a estrutura das pastas pode vir a se tornar complexa e de difícil manutenção. A introdução da análise de sensibilidade envolve criar sub-pastas dentro do hospital, o que pode complicar ainda mais a organização e a compreensão do sistema.

Outro desafio decorrente desse método de organização são as numerosas alterações registradas pelo sistema de controle de versão, como o Git, após a execução da análise de sensibilidade.

Para resolver isso, uma das possíveis soluções seria necessário incluir um sistema de banco de dados na aplicação. Para isso, seria necessário aproveitar a lista de funções de cada arquivo e elaborar um diagrama apropriado para substituí-los. Felizmente, a separação de responsabilidades feita neste trabalho é crucial para facilitar a inclusão de um banco de dados, já que a maioria das mudanças seria realizada no arquivo “db.py”.

5.1.2. Tradução do código do modelo.

O código atual em C do modelo foi gerado a partir do código original que estava em MATLAB, conforme detalhado no capítulo de Atividades Realizadas. O código gerado por máquina não oferece boa legibilidade, dificultando a introdução de novos recursos que envolvem alterações no modelo.

Para corrigir esse problema, seria necessário traduzir o código atual em C para uma linguagem mais legível e adotar padrões de projeto adequados, visando simplificar a estrutura do código e facilitar futuras mudanças. Se essa tarefa for realizada no futuro, recomenda-se adaptar o modelo para a linguagem Python, fazendo uso de bibliotecas como Numpy e SciPy, visto que grande parte do código do modelo envolve manipulação de matrizes. Essa pode ser considerada a atividade mais crucial, e também a mais difícil de ser feita.

5.1.3. Uso de dados reais.

Para realizar essa atividade, seria fundamental estabelecer uma parceria com uma instituição de saúde para testar as funcionalidades da ferramenta em ambientes reais. Com a presença de testes em cenários reais, seria possível levar o projeto em direção a cenários mais autênticos e representativos dos presentes em ambientes reais. Além disso, seria necessário adequar a ferramenta às especificações e complexidades de um ambiente real.

6. Referências

- [1] Assis, M. M. A., & Jesus, W. L. A. de .. (2012). Acesso aos serviços de saúde: abordagens, conceitos, políticas e modelo de análise. *Ciência & Saúde Coletiva*, 17(11), 2865–2875. <https://doi.org/10.1590/S1413-81232012001100002>
- [2] CEARÁ. Secretaria da Saúde do Estado. Nota técnica: fluxo de acesso às cirurgias eletivas. 2020. Disponível em: <https://www.saude.ce.gov.br/wp-content/uploads/sites/9/2018/06/nota_tecnica_fluxo_acesso_cirurgias_eletivas_14_10_2020.pdf>. Acesso em: abril de 2023
- [3] Bishop, Christopher (2006). *Pattern Recognition and Machine Learning* (em inglês). [S.l.]: Springer. ISBN 0-387-31073-8
- [4] Goldberg, D.E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. EUA: Addison-Wesley. 0-201-15767-5
- [5] GONZÁLEZ, RENÉ ; Vellasco, Marley ; FIGUEIREDO, KARLA . Resource optimization for elective surgical procedures using quantum-inspired genetic algorithms. In: the Genetic and Evolutionary Computation Conference, 2019, Prague. *Proceedings of the Genetic and Evolutionary Computation Conference on - GECCO '19*. New York: ACM Press, 2019. p. 777-786.
- [6] ROCHA, Leonardo Luiz Andrade. Interface para otimização de recursos de procedimentos cirúrgicos eletivos utilizando algoritmos genéticos com inspiração quântica. 2022. 45 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Computação) - Pontifícia Universidade Católica do Rio de Janeiro, Rio de Janeiro.
- [7] G. Zhang and Gexiang, “Quantum-inspired evolutionary algorithms: a survey and empirical study,” *J. Heuristics*, vol. 17, no. 3, pp. 303–351, Jun. 2011.
- [8] L. R. da Silveira, R. Tanscheit, and M. Vellasco, “Quantum inspired evolutionary algorithm for ordering problems,” *Expert Syst. Appl.*, vol. 67, pp. 71–83, Jan. 2017.
- [9] Python Software Foundation. Referência da linguagem Python, versão 3.12.0. Disponível em: <https://docs.python.org/pt-br/3/reference/index.html>. Acesso em novembro de 2023.

- [10] Flask (2010). Flask Documentation. Acesso em abril de 2023.
<https://flask.palletsprojects.com/>
- [11] CTypes. Ctypes Documentation. Acesso em novembro de 2023.
<https://docs.python.org/3/library/ctypes.html>
- [12] Jinja. Jinja Documentation. Acesso em novembro de 2023.
<https://jinja.palletsprojects.com/en/3.1.x/>
- [13] Tsonis, A.C., Smith, C.A., & Thomsen, S.C. (2023). The Future of Sensitivity Analysis: An essential discipline for systems modeling and policy support. *Nature Communications*, 14(1), 1-12.